

Action Imitation

Action Imitation

Introduction

Operation Steps

Preparatory Steps

Getting Started

Continuous Motion Learning and Performance

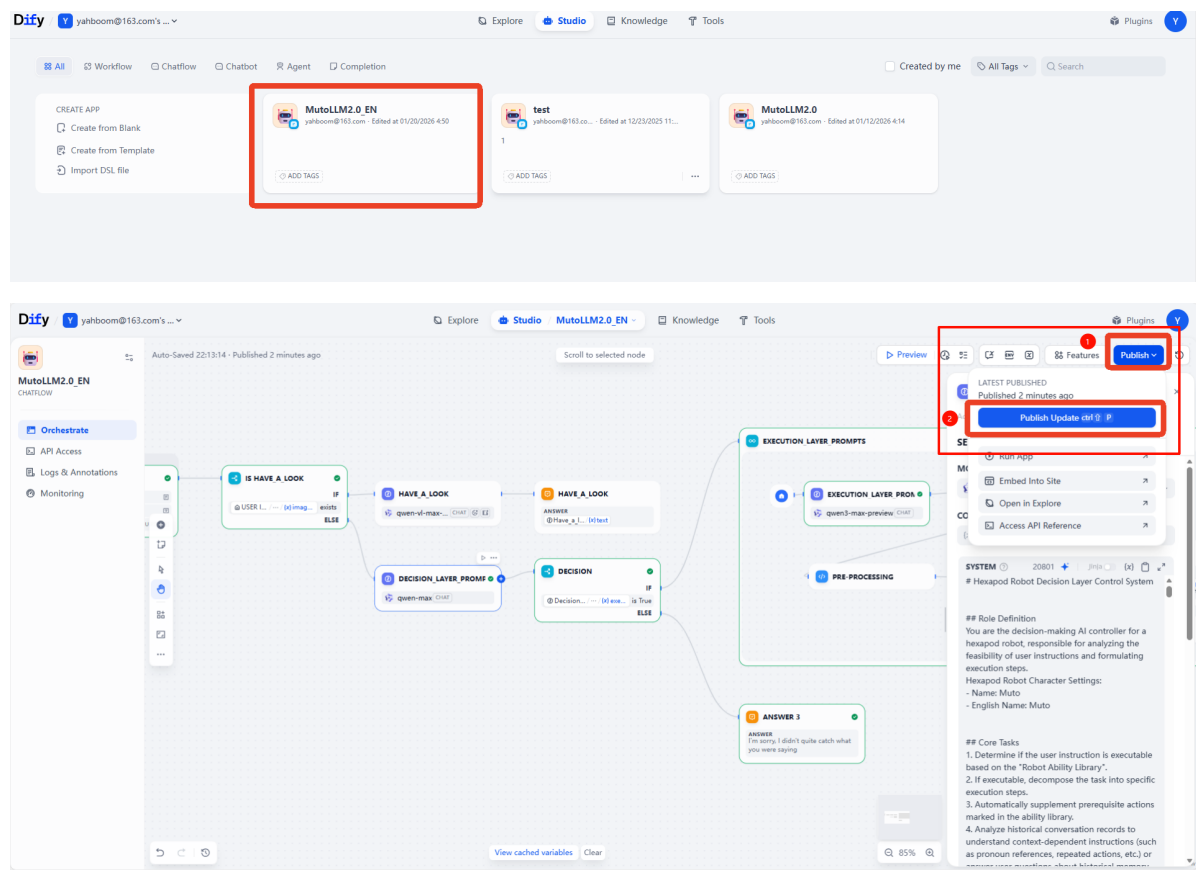
Introduction

In this tutorial, we will demonstrate how to teach Muto actions by chatting with it through the Dify interface.

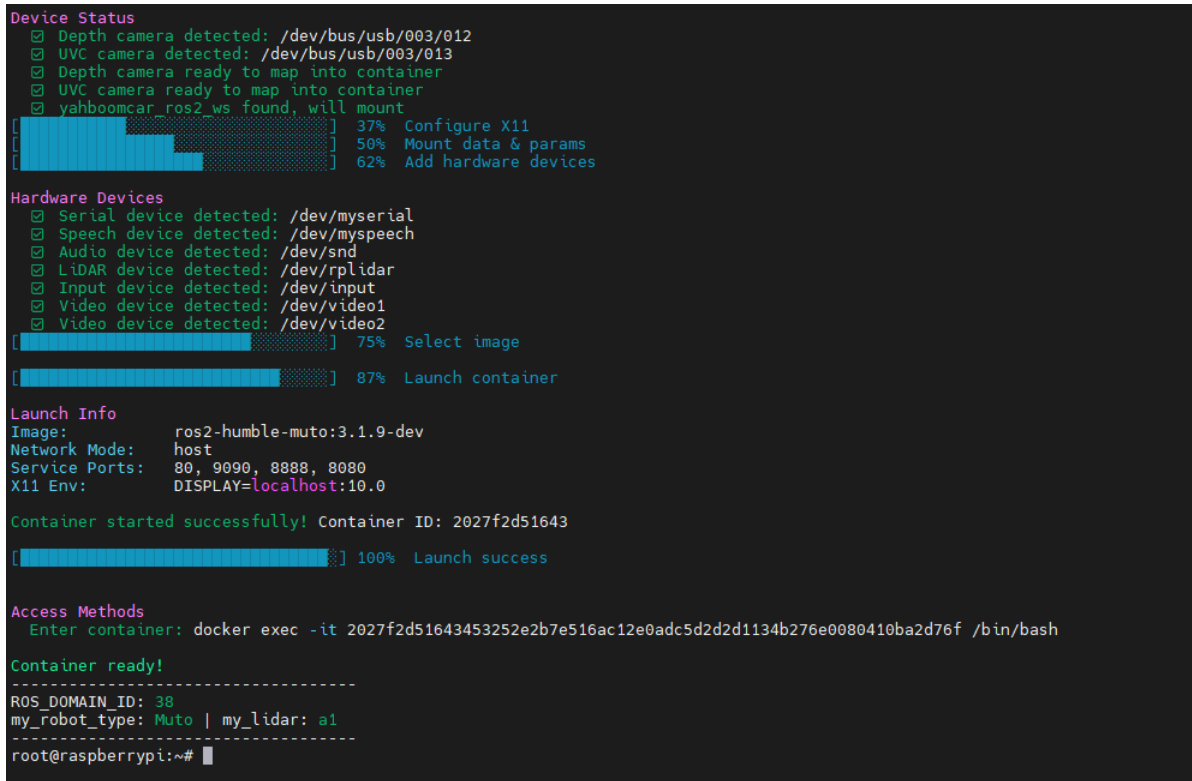
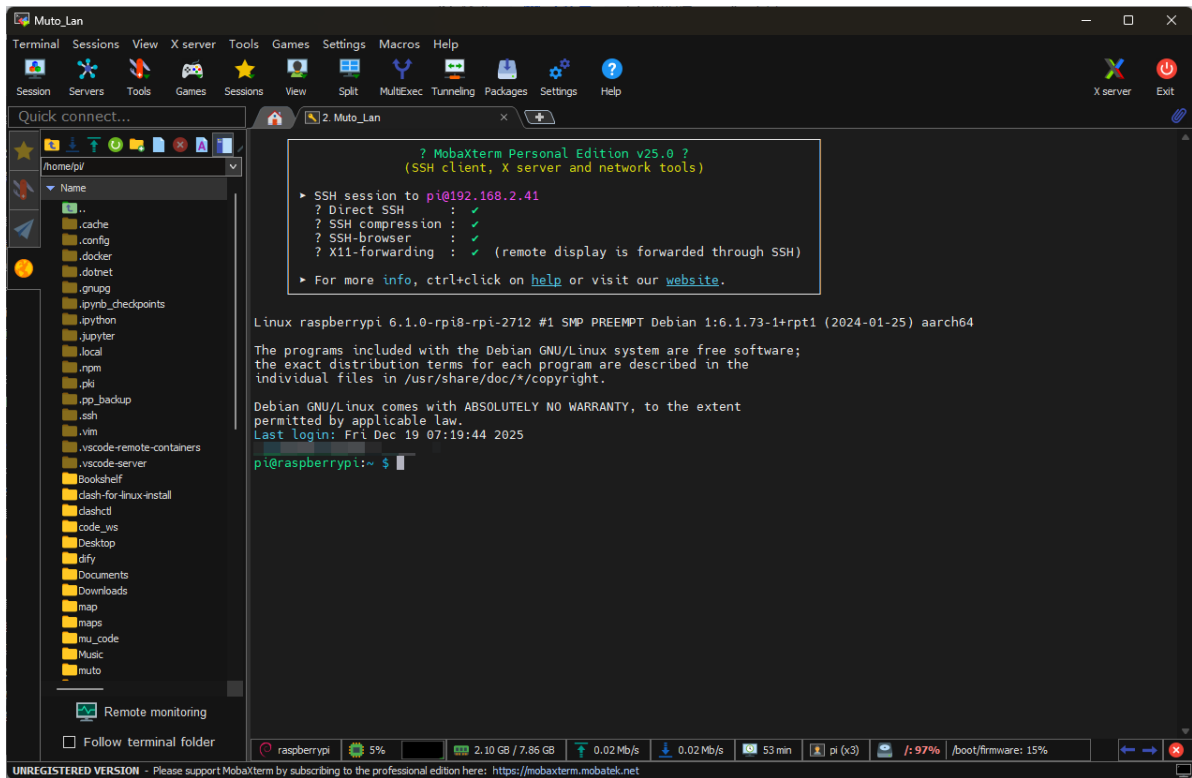
Operation Steps

Preparatory Steps

This lesson requires the Dify terminal to be running, ensuring Dify is functioning correctly and all configuration items are set up as described in previous sections.



This lesson requires the MutoRS to be powered on. Following the instructions in the previous sections... Starting a large model terminal in Docker



```
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239218] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

Getting Started

The commands we'll use in this lesson are as follows:

I'll teach you a Tai Chi routine, and after you've learned it, perform it for me twice.

```
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
[muto_controller_node-1] Mode: Sync (Blocking)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "Let me observe the surroundings and compose a poem for you.", "have_a_look(user_query": "Analyze the main features of the current environment and create a four-line poem based on these features.")"}
[muto_controller_node-1] [Muto reply] Let me observe the surroundings and compose a poem for you.
[muto_controller_node-1] {"event": "execute_commands", "success": true, "executed_count": 3, "failed_command_id": null, "duration_ms": 12373}
[muto_controller_node-1] Mode: Async (Background)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "Office chairs swivel, feet up high.", "head_move(level=2, direction": "left", "stretch()")"}
[muto_controller_node-1] [Muto reply] Office chairs swivel, feet up high.
[muto_controller_node-1] Mode: Async (Background)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "Screens glow with tasks to assign.", "spin_in_place()", "warm_up_squat()")"}
[muto_controller_node-1] [Muto reply] Screens glow with tasks to assign.
[muto_controller_node-1] move(0, 0, 0) test
[muto_controller_node-1] Mode: Async (Background)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "Boxes stack near the desk's edge", "point_left_front()", "stretch()")"}
[muto_controller_node-1] [Muto reply] Boxes stack near the desk's edge
[muto_controller_node-1] Plan Data: {"robot_speak(text": "Quiet focus in the workspace's hush.", "point_right_front()", "curl_up()")"}
[muto_controller_node-1] [Muto reply] Quiet focus in the workspace's hush.
[muto_controller_node-1] Unhealthy components detected: [ 'camera_system' ]
[muto_controller_node-1] Mode: Async (Background)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "Great! I'm ready to learn the Tai Chi routine.")"}
[muto_controller_node-1] [Muto reply] Great! I'm ready to learn the Tai Chi routine.
[muto_controller_node-1] Mode: Sync (Blocking)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "I am now starting to learn the Tai Chi routine.", "start_continuous_action_learning(sequence_name": "Tai Chi")"}
[muto_controller_node-1] [Muto reply] I am now starting to learn the Tai Chi routine!
[muto_controller_node-1] 正在进入学习准备姿态...
[muto_controller_node-1] 学习准备姿态设置完成
[muto_controller_node-1] 开始动作: 已就绪
[muto_controller_node-1] 已记录动作 1: [0, -35, -20, 0, -35, -20]
[muto_controller_node-1] 已记录动作 2: [0, -34, -19, 0, -35, -20]
[muto_controller_node-1] 已记录动作 3: [0, -34, -19, 0, -35, -20]
[muto_controller_node-1] 已记录动作 4: [0, -34, -19, 0, -35, -20]
[muto_controller_node-1] 已记录动作 5: [0, -34, -19, 0, -35, -20]
[muto_controller_node-1] 已记录动作 6: [0, -34, -19, 0, -35, -20]
[muto_controller_node-1] 已记录动作 7: [0, -59, 6, 0, -35, -20]
[muto_controller_node-1] 已记录动作 8: [0, -60, 6, 0, -60, -2]
[muto_controller_node-1] 已记录动作 9: [0, -40, -22, 0, -44, -26]
[muto_controller_node-1] 已记录动作 10: [-1, -79, 17, 0, -44, -26]
[muto_controller_node-1] 已记录动作 11: [0, -87, 23, 0, -44, -26]
[muto_controller_node-1] 已记录动作 12: [0, -87, 23, 0, -44, -26]
[muto_controller_node-1] 已记录动作 13: [0, -87, 23, 0, -44, -26]
[muto_controller_node-1] 已记录动作 14: [0, -87, 23, 0, -44, -26]
[muto_controller_node-1] 已记录动作 15: [0, -87, 23, 0, -44, -26]
[muto_controller_node-1] 停止动作学习模式...
[muto_controller_node-1] 学习模式已停止
[muto_controller_node-1] 连续动作序列已保存到: action_data/Tai Chi.json
[muto_controller_node-1] {"event": "execute_commands", "success": true, "executed_count": 3, "failed_command_id": null, "duration_ms": 33786}
[muto_controller_node-1] Mode: Async (Background)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "I've finished learning the Tai Chi routine!")"}
[muto_controller_node-1] [Muto reply] I've finished learning the Tai Chi routine!
[muto_controller_node-1] Mode: Async (Background)
[muto_controller_node-1] Plan Data: {"robot_speak(text": "Watch me perform Tai Chi!", "play_action_sequence(filename": "Tai Chi")"}
[muto_controller_node-1] [Muto reply] Watch me perform Tai Chi!
[muto_controller_node-1] 已加载动作序列: 15 个动作
[muto_controller_node-1] 正在进入学习准备姿态...
```

When we see `start_continuous_action_learning` output from the backend, Muto will enter learning mode.

At this point, we can guide Muto to learn the movements by moving its two forelimbs. Muto will continuously record ten movements.

After learning, Muto's forelimbs will quickly return to their original positions.

The return speed is fast, please be careful.

Then perform the newly learned "Tai Chi" sequence twice.

Continuous Motion Learning and Performance

The motions learned by Muto are persistently saved on the motherboard. Multiple different motion files can be stored on Muto's motherboard, each with a different name.

Let's teach Muto another set of motions and then have it perform both sets.

The screenshot displays the Dify AI interface, specifically the 'Orchestrate' tab for a workflow named 'MutoLLM2.0_EN'. The workflow is a linear sequence of steps: 'BROADCAST', 'IF/ELSE 4', 'PARSER', and 'END'. The 'IF/ELSE 4' step is expanded, showing a logic flow where a 'Get JSON' action leads to a 'result' variable, which then branches into 'True' and 'False' paths. The 'True' path leads to an 'ACTION' step, and the 'False' path leads to an 'END' step. The 'PARSER' step is also expanded, showing a 'CONTENT' variable. The 'END' step is labeled 'Workflow End'.

On the right side of the interface, there is a 'PREVIEW' window showing the results of the workflow execution. It contains two sections, each with a 'Workflow Process' and a list of actions and their results. The first section shows the results of teaching Muto a Tai Chi routine, and the second section shows the results of teaching Muto a Wing Chun routine. Both sections indicate successful execution and provide details about the actions performed, such as 'Start learning action', 'Reply to user', and 'Play action sequence'.

Observing Muto's behavior, we can see that both sets of motions play correctly.

Our motion files are permanently saved on Muto's motherboard, so the motions will still exist after a system restart.